Tavan Bhatia (tb568), Nicholas Gavalas (nag72), Cameron Harvey (cnh43)

MAE 3780 Final Report: The Sticky Bandits

Robot Design and Strategy Overview:

Our robot was designed around a simple, reliable strategy: move quickly to a high-value region of the board, collect cubes efficiently, and then sweep through the field in a repeatable pattern.

Mechanically, the robot used a box-style chassis that maximized the allowed bounding box while leaving enough open space to take in and carry cubes. This large rectangular frame helped guide cubes into the robot as it moved, while the side rails and top-corner flanges helped retain cubes during sharp turns and collisions. The wheels and color sensor were placed in protected locations so that impacts with other robots or cubes would not easily interrupt movement or path detection. The overall mechanical design avoided complicated moving mechanisms and instead focused on being durable, fast, and able to physically control space on the board.

Electrically, the robot relied primarily on a color sensor to detect the color of the surface underneath it. This allowed the robot to identify when it crossed into a new region of the board and trigger the next stage of its movement algorithm. The color sensor gave us direct feedback

from the floor, which was important because our strategy depended on recognizing board boundaries rather than only running motors for fixed amounts of time.

In software, the robot was programmed to begin by traveling toward the corner between the yellow and blue regions along the edge of the board. Once it reached that area, it would turn to align itself with the yellow-blue border. From there, it would move in a serpentine pattern down the halfway line, sweeping back and forth to collect cubes. After completing one pass, the robot would return partway back while continuing the serpentine motion, and then begin a spiral-like path around the center that expanded outward. Ideally, this strategy allowed the robot to collect cubes from both edge and center regions while avoiding unnecessary complexity.

Design Process Reflection:

From the first milestone, our group decided that our main design philosophy would be simplicity and reliability. We wanted to build a robot with “no gimmicks,” meaning that instead of spending time on a complicated cube-collection mechanism, we would focus on making a straightforward design that could execute its strategy consistently. Because the competition involved directly competing against other robots, we believed that many teams might lose time trying to perfect elaborate mechanisms. We instead wanted to focus on the decisive parts of the challenge, such as making sure the robot moved quickly, the color sensor code was tuned well, and the chassis could survive contact with other robots.

` Once the design was finalized, we laser cut the chassis pieces and assembled the robot. After the mechanical structure was built, most of our testing focused on getting the movement code and color sensor behavior to work reliably together. One major roadblock was that the color sensor behaved differently depending on the lighting conditions. When the robot was plugged into a laptop, the sensor readings were different from when it was running on battery power, largely because of the sensor LED and surrounding light conditions. To make testing more consistent, we turned off the LED pin entirely and used a phone flashlight at a brightness similar to the light level produced when the robot was running on battery power.

We also noticed that blemishes and small inconsistencies on the floor could cause noisy color readings, so we adjusted the software to filter out small changes and avoid false triggers. We chose not to use the QTI sensor because we wanted to keep the robot as focused and simple as possible. At first, we had our robot simply detect its starting color and then move to a new state if it detected a changed color. This approach gave way to lots of inconsistency as far as the imperfect ground. To account for this, we had our “starting color” recognized as the average of the first ticks rather than the very first, and we compared that to a moving average of the last ticks. This way, one darker spot wouldn't throw us off. This strategy helped us avoid any false stops on yellow, but on blue the code was still iffy. We theorized that this came not from

our blemish issue, but from our idea of the color “changing” that was hardwired into the code. We realized that the output from the sensor was in a higher range on the blue section, so the magnitude of blemishes was higher too. In such, we set a threshold color reading which changed the cut off point for a “new color” to be detected. To this end, the higher variance in the blue range was accounted for.

Competition Analysis:

Our robot performed up to our expectations by the end of the tournament; as the round robin started off rocky with three loses, but then concluded with three consecutive wins. Our main struggle was getting the color sensor to properly trigger our movement algorithm upon a color change as mentioned above. Other than our color sensor issue, we also would be at a disadvantage if the other team's robot was heading to the same corner of the board as us or was heading closer to the center. Our robot would either get pushed off or blocked when trying to collect cubes. In hindsight it may have proven useful to have a switch that could change what corner we went to first. In our last few rounds, the color sensor began to trigger and we were less interfered with by the other robot, so we actually managed to execute the entire algorithm and successfully win those rounds. Overall our process was strong, but we didn't anticipate the negative impact that a competing robot and a sporadic sensor had on our total performance.

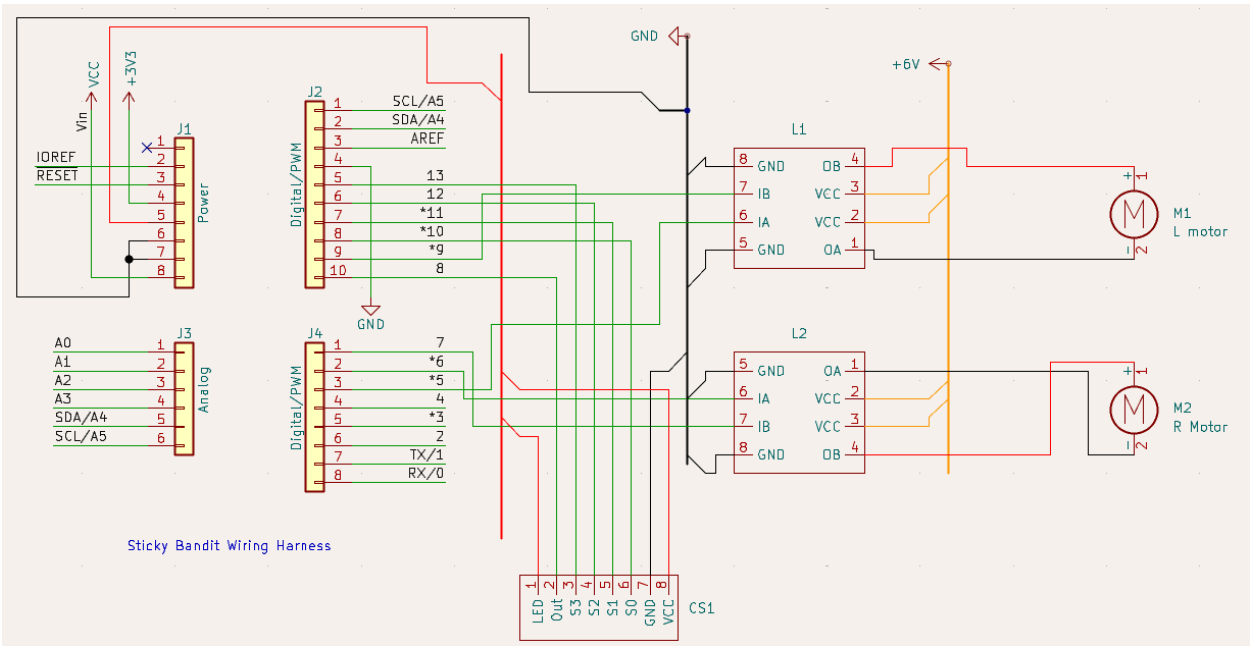
Conclusions:

Reflecting on our project, there are a few different things that we may have done differently with our design. First, as we mentioned above, it could’ve been a good idea to code two algorithms and have a physical switch to be able to change between them, so we could adapt to the starting movement of the other robot. Secondly, given our sensor issue it may have been best to just hardcode the entire movement sequence and not rely on sensors to perform well. Ultimately, we could have spent time on purely tuning the full movement and we would’ve run into way less issues with sensor output. On this same note, our advice to students next year would be to not overcomplicate your robot. This is more than just a specific change to make in the algorithm but something to start off addressing as a first principle in your code. One example of change this may result in would be taking the seemingly less optimal start going straight to the center, then rotating 90 degrees and scooping half the cubes before doing a 180 and returning. Though this seems worse than going to a corner and having a pass at all of the cubes, the fact that almost every team went to a corner meant that it came with a collision and a lot of uncertainty. If you find one small niche and perform it well then you have a good chance of going far in the competition.

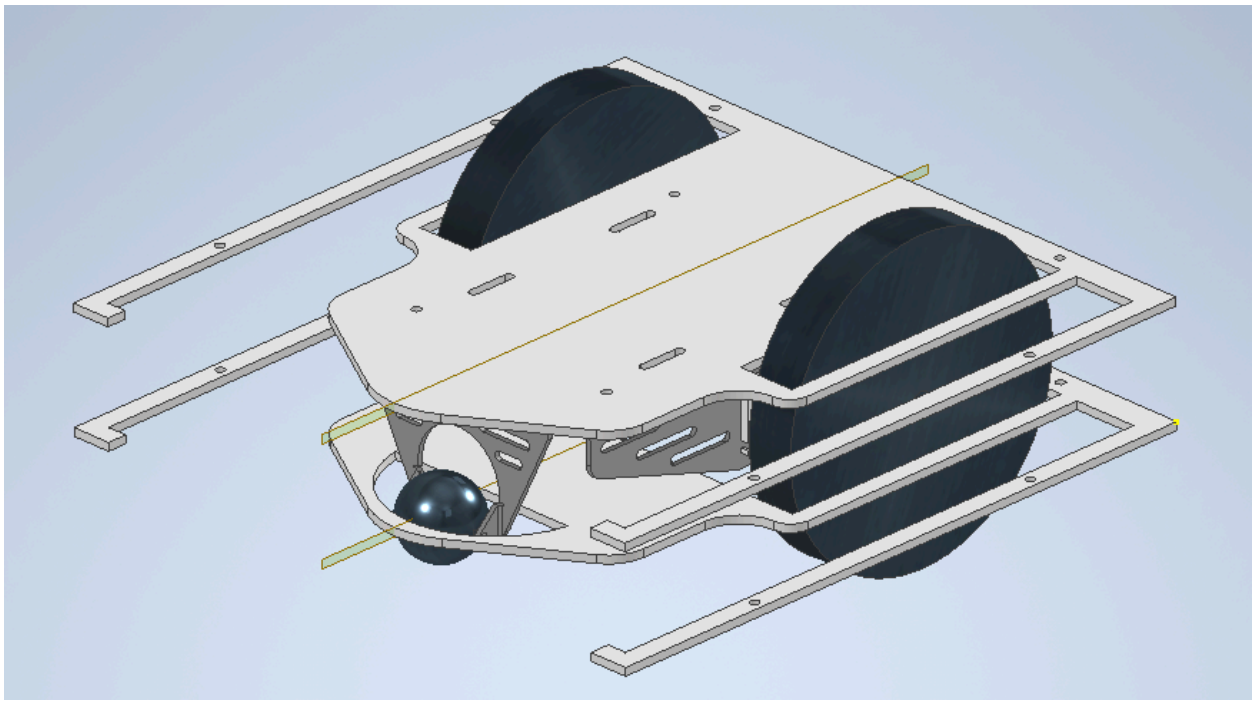
Appendix A: Bill of Materials

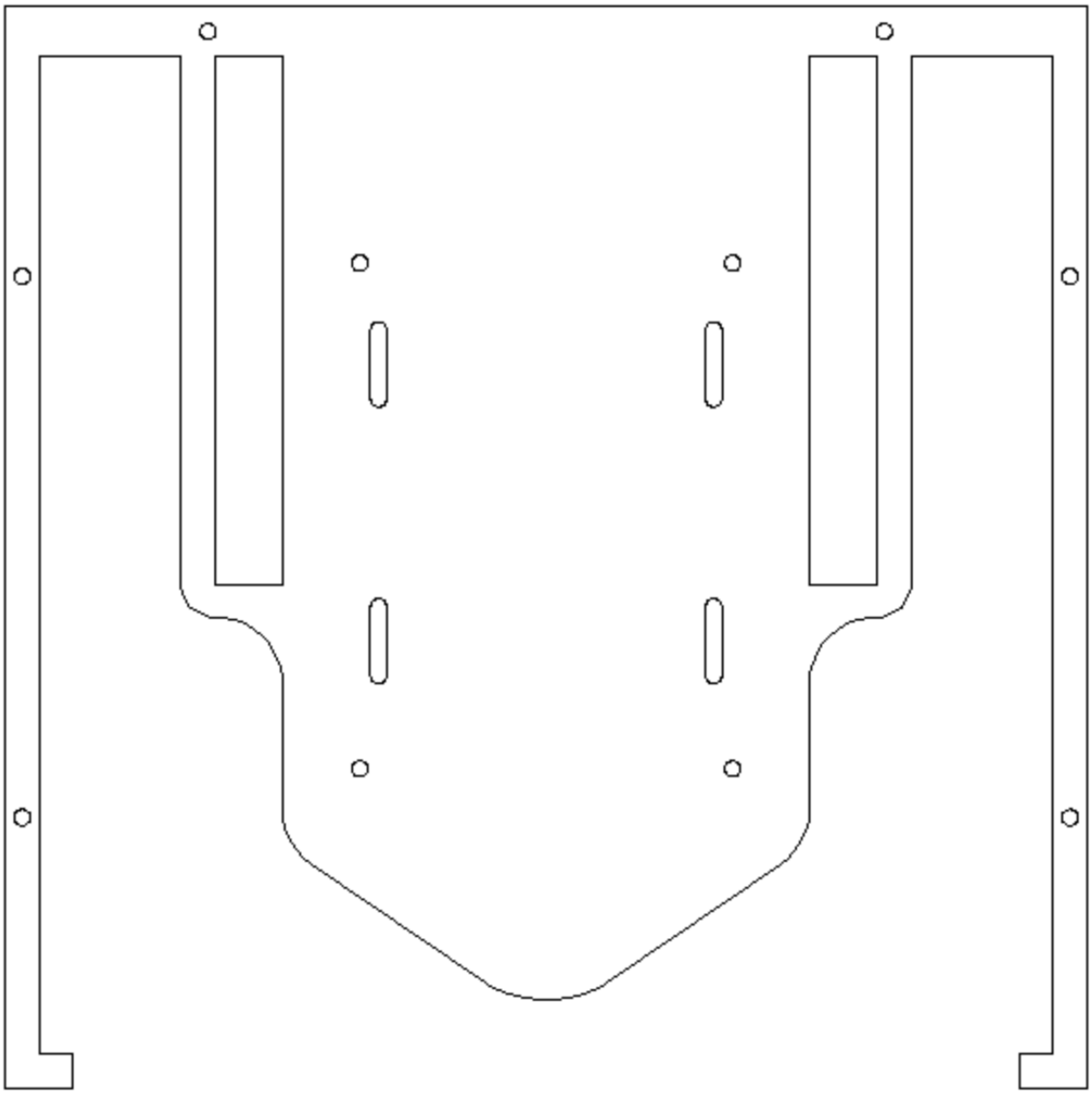
Purchased Components:	Cost
Acrylic Frame Bottom (1 Sheet, 1 Part, 80 in)	$ 9.50
Acrylic Frame Bottom (1 Sheet, 1 Part, 92 in)	$ 10.10
Total Cost	$ 19.60

Appendix B: Circuit Diagram

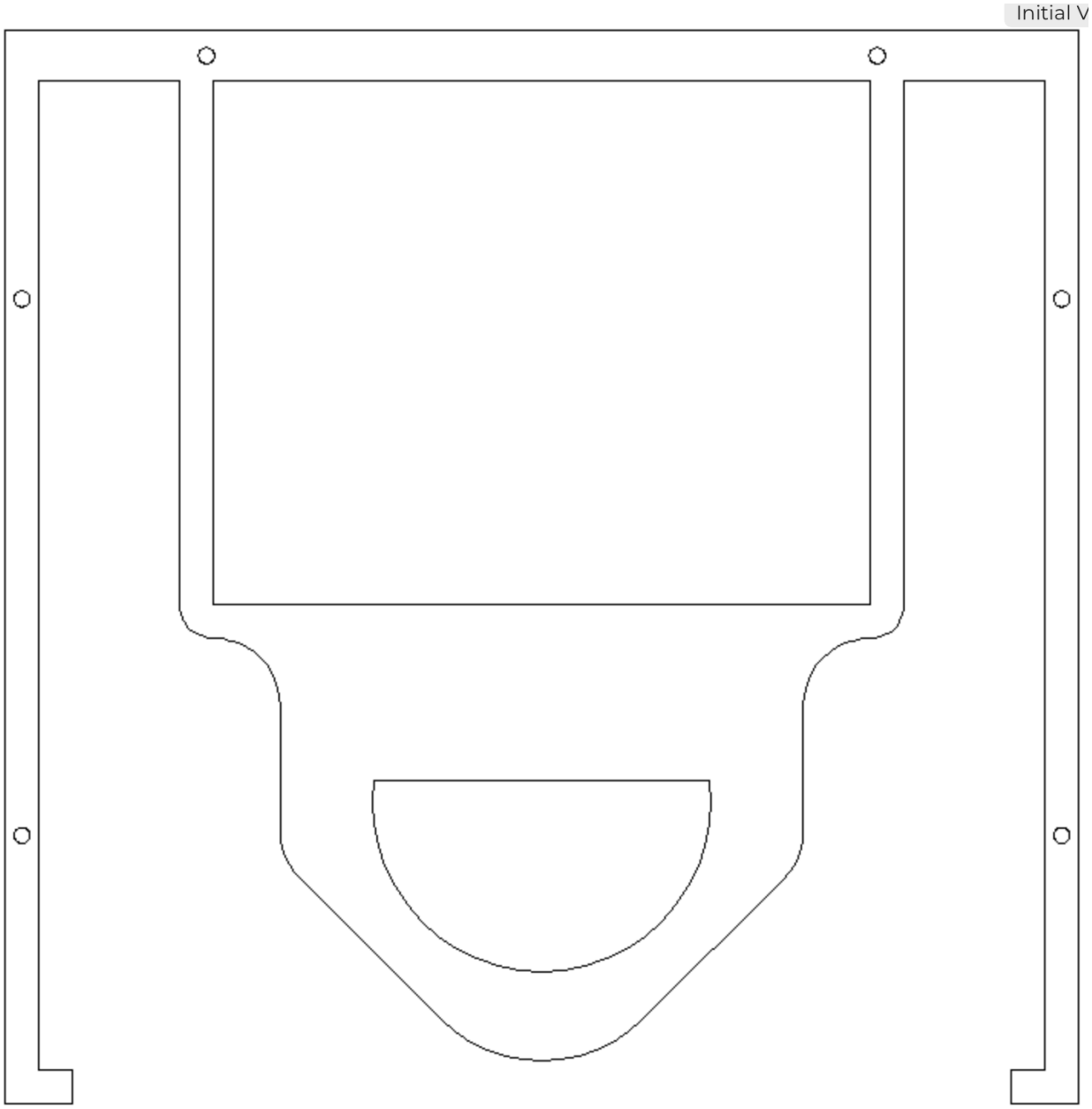


Appendix C: CAD Files and Drawings



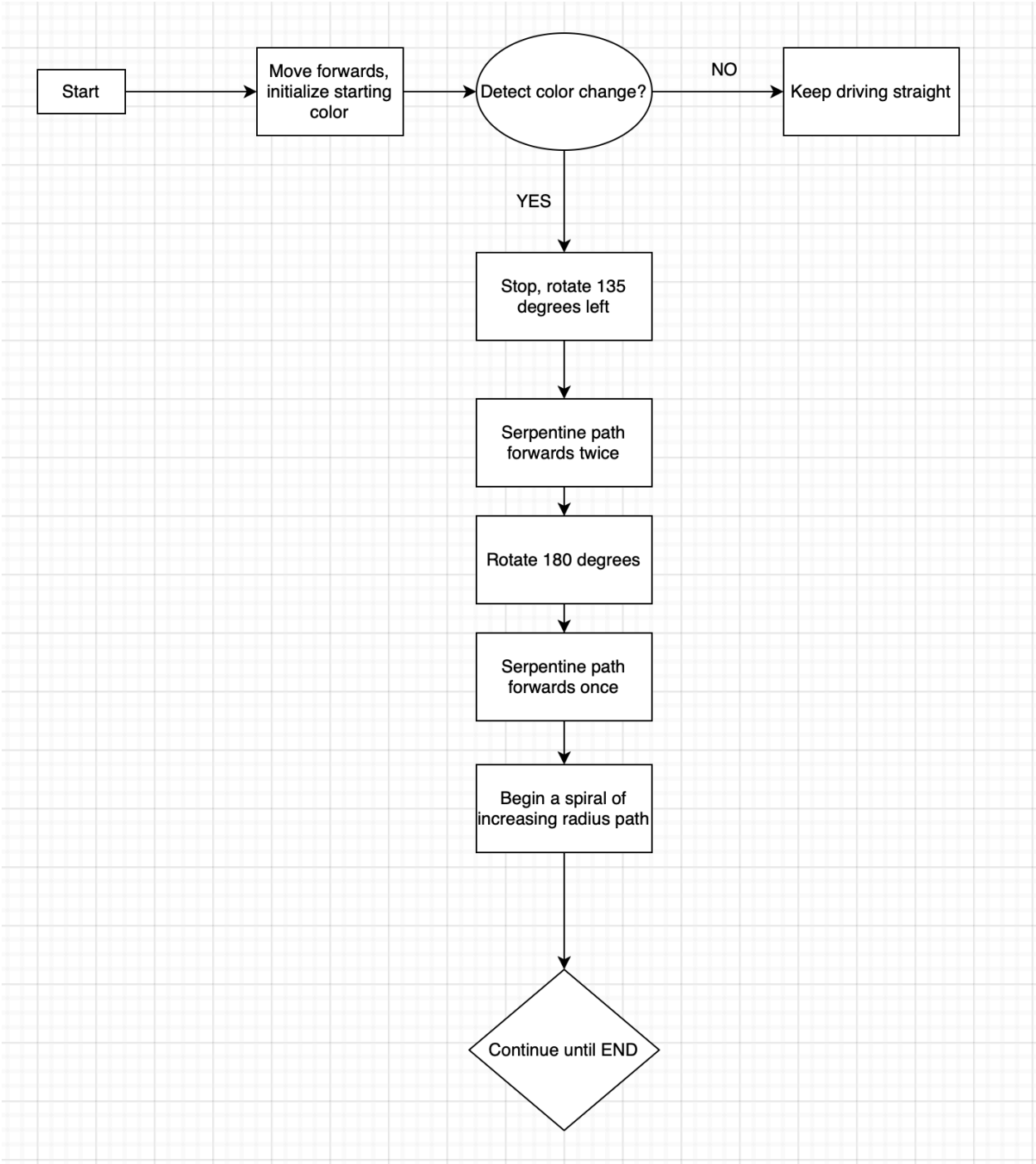


Top plate (185 g)



Bottom plate (115 g)

Appendix D: Flowchart



Hard coded nature of our robot is clearly visible.

Appendix E: Code

```
/*
  Combined behavior:

  1. Calibrate color using the average of the first NUM_TICKS readings.
  2. Drive straight until the average of the latest NUM_TICKS readings
     changes enough from the initial average.
  3. Stop.
  4. Turn left about 135 degrees.
  5. Run the movement pattern from the serpentine file forever.

  Color sensor:
    S0  = D13 / PB5
    S1  = D12 / PB4
    S2  = D11 / PB3
    S3  = D10 / PB2
    OUT = D8  / PB0

  Motor wiring:
    Left motor  -> D5 (PD5) and D9 (PB1)
    Right motor -> D6 (PD6) and D7 (PD7)
*/

#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>

// -------------------- COLOR TUNING --------------------
#define NORMAL_COLOR_CHANGE_AMOUNT 200
#define HIGH_COLOR_CHANGE_AMOUNT 300
#define NUM_TICKS 7

// -------------------- TURN TUNING --------------------
const unsigned int TURN_135_MS = 675;

// -------------------- SERPENTINE TUNING --------------------
const unsigned int SERPENTINE_TIME_MS = 10000;

// Tune this until the robot turns about 180 degrees.
const unsigned int TURN_180_MS = 1000;
```

```
// Big LEFT curve: left wheel slow, right wheel fast
const uint8_t LEFT_CURVE_LEFT_SPEED = 70;
const uint8_t LEFT_CURVE_RIGHT_SPEED = 255;
const unsigned int LEFT_CURVE_TIME_MS = 1000;

// Big RIGHT curve: left wheel fast, right wheel slow
const uint8_t RIGHT_CURVE_LEFT_SPEED = 255;
const uint8_t RIGHT_CURVE_RIGHT_SPEED = 70;
const unsigned int RIGHT_CURVE_TIME_MS = 1300;

// -------------------- SPIRAL TUNING --------------------
const unsigned long SPIRAL_TIME_MS = 30000;
const uint8_t SPIRAL_OUTER_SPEED = 255;
const uint8_t SPIRAL_INNER_START_SPEED = 40;
const uint8_t SPIRAL_INNER_END_SPEED = 230;

// -------------------- DELAY --------------------
void wait_ms(int x) {
  for (int i = 0; i < x; i++) {
    _delay_ms(1);
  }
}

void custom_delay_ms(unsigned int ms) {
  for (unsigned int i = 0; i < ms; i++) {
    _delay_ms(1);
  }
}

// -------------------- INIT --------------------
void initSystem(void) {
  // Motor pins
  DDRD = DDRD | 0b11100000;  // D5, D6, D7 outputs
  DDRB = DDRB | 0b00111110;  // D9, D10, D11, D12, D13 outputs

  // Color sensor OUT pin
  DDRB = DDRB & 0b11111110;  // D8 input

  // Color sensor frequency scaling:
```

```
  // S0 = HIGH, S1 = LOW
  PORTB = PORTB | 0b00100000;  // S0 high
  PORTB = PORTB & 0b11101111;  // S1 low

  // Red filter:
  // S2 = LOW, S3 = LOW
  PORTB = PORTB & 0b11110011;  // S2 low, S3 low

  // Timer1 for measuring color pulse width
  TCCR1A = 0b00000000;
  TCCR1B = 0b00000010;
  TCNT1 = 0;

  // Timer0 for PWM on D5 and D6 during serpentine
  TCCR0A = 0b00000011;
  TCCR0B = 0b00000011;
}

// -------------------- MOTORS --------------------
void halt(void) {
  // Disconnect Timer0 PWM from D5/D6
  TCCR0A = 0b00000011;

  PORTD = PORTD & 0b00011111;
  PORTB = PORTB & 0b11111101;
}


void forward(void) {
  // Straight forward without PWM
  TCCR0A = 0b00000011;

  PORTB = PORTB & 0b11111101;  // D9 low
  PORTD = PORTD & 0b01111111;  // D7 low

  PORTD = PORTD | 0b01100000;  // D5 high, D6 high
}

void driveForwardPWM(uint8_t leftSpeed, uint8_t rightSpeed) {
  // Backward pins LOW
```

```
  PORTD = PORTD & 0b01111111;
  PORTB = PORTB & 0b11111101;

  OCR0B = leftSpeed;   // Left Forward, D5 / PD5
  OCR0A = rightSpeed;  // Right Forward, D6 / PD6

  // Connect Timer0 to D5 and D6 in Fast PWM mode
  TCCR0A = 0b10100011;
}

void turnLeft135(void) {
  halt();
  wait_ms(100);

  // Timer0 disconnected so pins can be controlled manually
  TCCR0A = 0b00000011;

  // Spin left:
  // left motor backward = D9 / PB1 HIGH
  // right motor forward = D6 / PD6 HIGH
  PORTD = PORTD & 0b00011111;
  PORTB = PORTB & 0b11111101;

  PORTB = PORTB | 0b00000010;
  PORTD = PORTD | 0b01000000;

  wait_ms(TURN_135_MS);

  halt();
  wait_ms(100);
}

void turn180(void) {
  halt();
  custom_delay_ms(100);

  // Timer0 disconnected so pins can be controlled manually
  TCCR0A = 0b00000011;

  // Spin left:
```

```
  // left motor backward = D9 / PB1 HIGH
  // right motor forward = D6 / PD6 HIGH
  PORTD = PORTD & 0b00011111;
  PORTB = PORTB & 0b11111101;

  PORTD = PORTD | 0b01000000;
  PORTB = PORTB | 0b00000010;

  custom_delay_ms(TURN_180_MS);

  halt();
  custom_delay_ms(100);
}

// -------------------- COLOR SENSOR --------------------
unsigned int readColor(void) {
  unsigned int count = 0;

  while ((PINB & 0b00000001) == 0) {
    count++;
    if (count > 60000) {
      return 0;
    }
  }

  count = 0;

  while ((PINB & 0b00000001) != 0) {
    count++;
    if (count > 60000) {
      return 0;
    }
  }

  TCNT1 = 0;

  while ((PINB & 0b00000001) == 0) {
    if (TCNT1 > 60000) {
      break;
    }
```

```
  }

  return TCNT1;
}

unsigned int getAverageColor(void) {
  unsigned int currentColor = 0;
  unsigned int totalColor = 0;

  for (int i = 0; i < NUM_TICKS; i++) {
    currentColor = readColor();
    totalColor = totalColor + currentColor;
    wait_ms(20);
  }

  return totalColor / NUM_TICKS;
}

// -------------------- SERPENTINE MOVEMENT --------------------
void runSerpentine(void) {
  driveForwardPWM(LEFT_CURVE_LEFT_SPEED, LEFT_CURVE_RIGHT_SPEED);
  custom_delay_ms(LEFT_CURVE_TIME_MS);

  driveForwardPWM(RIGHT_CURVE_LEFT_SPEED, RIGHT_CURVE_RIGHT_SPEED);
  custom_delay_ms(RIGHT_CURVE_TIME_MS);

  halt();
}

void runSpiralFor30Seconds(void) {
  const unsigned int delayStepMs = 50;
  const unsigned int totalSteps = SPIRAL_TIME_MS / delayStepMs;

  for (unsigned int step = 0; step < totalSteps; step++) {
    unsigned int speedRange = SPIRAL_INNER_END_SPEED -
SPIRAL_INNER_START_SPEED;
    uint8_t currentInnerSpeed = SPIRAL_INNER_START_SPEED + ((speedRange *
step) / (2 * totalSteps));

    driveForwardPWM(currentInnerSpeed, SPIRAL_OUTER_SPEED);
```

```
    custom_delay_ms(delayStepMs);
  }

  halt();
}

void runSerpentinePatternForever(void) {
  while (1) {
    runSerpentine();
    runSerpentine();
    turn180();
    runSerpentine();
    runSpiralFor30Seconds();
  }
}

// -------------------- ARDUINO SETUP / LOOP --------------------
int main(void) {
  initSystem();
  Serial.begin(9600);

  halt();
  wait_ms(50);

  Serial.println("Starting combined program");

  while (1) {
    static unsigned int initialAverageColor = 0;
    static unsigned int colorChangeAmount = 0;
    static int calibrated = 0;
    static int colorHit = 0;

    unsigned int averageColor = 0;
    unsigned int difference = 0;

    if (calibrated == 0) {
      halt();
      wait_ms(500);

      initialAverageColor = getAverageColor();
```

```
    if (initialAverageColor >= 2000) {
      colorChangeAmount = HIGH_COLOR_CHANGE_AMOUNT;
    } else {
      colorChangeAmount = NORMAL_COLOR_CHANGE_AMOUNT;
    }

    Serial.print("Initial average color: ");
    Serial.println(initialAverageColor);

    Serial.print("Color change threshold: ");
    Serial.println(colorChangeAmount);

    calibrated = 1;
    wait_ms(300);
  }

  if (colorHit == 0) {
    forward();

    averageColor = getAverageColor();
    difference = abs((int)averageColor - (int)initialAverageColor);

    Serial.print("Initial average: ");
    Serial.print(initialAverageColor);
    Serial.print(" Current average: ");
    Serial.print(averageColor);
    Serial.print(" Absolute difference: ");
    Serial.print(difference);
    Serial.print(" Threshold: ");
    Serial.println(colorChangeAmount);

    if (difference > colorChangeAmount) {
      Serial.println("Color change detected");
      colorHit = 1;

      halt();
      wait_ms(100);

      Serial.println("Turning left 135");
```

```
        turnLeft135();

        Serial.println("Starting serpentine pattern");
        runSerpentinePatternForever();
      }

      wait_ms(50);
    }
  }
}
```